MONITORIZACIÓN DE UN HUERTO

API REST



VNiVERSiDAD D SALAMANCA

Carlos Cuadrado Pérez-Rubín

Desarrollo de aplicaciones avanzadas

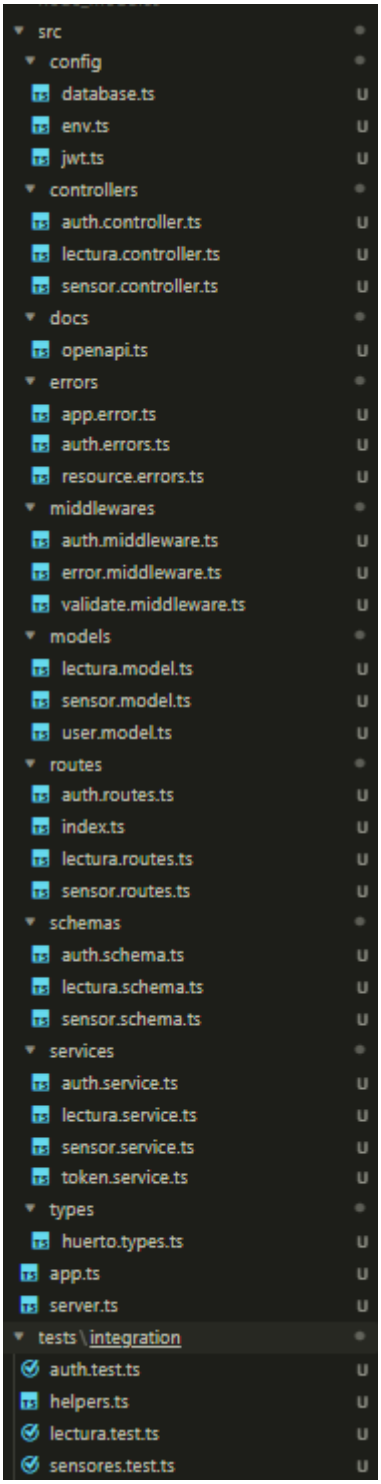
1. RESUMEN

El proyecto plantea una API REST que gestiona los sensores de un huerto y sus lecturas. Hay dos niveles de usuario: ***usuario***, que consulta sensores y registra y consulta lecturas y ***administrador***, que además de alta, modifica y elimina sensores. El servidor calcula si cada lectura es anómala al guardarla comparando su valor con el límite del sensor.

2. ARQUITECTURA

Cada petición recorre las mismas capas y cada capa tiene una única responsabilidad:

Capa	**Carpeta**	**Responsabilidad**
Rutas	**routes/**	Asocian cada endpoint con sus middlewares y su controlador
Middlewares	**middlewares/**	Autenticación, roles, validación de entrada y manejo de errores
Controladores	**controllers/**	Traducen la petición HTTP a una llamada al servicio y devuelven la respuesta
Servicios	**services/**	Lógica de negocio (registro, login, anomalías, filtros, paginación)
Modelos	**models/**	Esquemas de Mongoose y acceso a MongoDB
Apoyo	**schemas/, types/, errors/, config/, docs/**	Validación Zod, tipos, errores propios, configuración y OpenAPI



(figura 1 src/ y tests/integration/ desplegados)

app.ts U

src > app.ts > ...

```ts
import express, { Request, Response } from "express";
import router from "./routes/index.js";
import { notFoundHandler, globalErrorHandler } from "./middlewares/error.middleware.js";

import swaggerUi from "swagger-ui-express";
import { openapiSpec } from "./docs/openapi.js";

export const app = express();
app.disable("x-powered-by");

app.use("/api/docs", swaggerUi.serve, swaggerUi.setup(openapiSpec));

app.use(express.json());

app.get("/api/health", (_req: Request, res: Response) => {
  res.status(200).json({
    status: "success",
    data: {
      status: "ok",
      timestamp: new Date().toISOString(),
      uptime: process.uptime(),
    },
  });
});

app.use("/api", router);

app.use(notFoundHandler);
app.use(globalErrorHandler);

```

(figura 2, app.ts)

server.ts U

src > server.ts > ...

```ts
import { app } from "./app.js";
import { env } from "./config/env.js";
import { connectDB, disconnectDB } from "./config/database.js";

async function bootstrap() {
  await connectDB();

  const server = app.listen(env.port, (error?: Error) => {
    if (error) {
      console.error("No se pudo abrir el puerto:", error);
      process.exit(1);
    }
    console.log(`Servidor escuchando en http://localhost:${env.port}`);
  });

  const shutdown = () => {
    server.close(async () => {
      await disconnectDB();
      process.exit(0);
    });
  };

  process.on("SIGINT", shutdown);
  process.on("SIGTERM", shutdown);
}

bootstrap().catch((error) => {
  console.error("No se pudo arrancar el servidor:", error);
  process.exit(1);
});
```

(figura 3, server.ts)

app.ts configura Express (rutas y middlewares) y lo exporta sin arrancar el puerto, permitiendo testear la app con Supertest de forma aislada. server.ts es el único punto de entrada real: conecta a MongoDB, ejecuta app.listen() y gestiona el apagado seguro del servidor.

3. MODELO DE DATOS

3.1 Colecciones

users

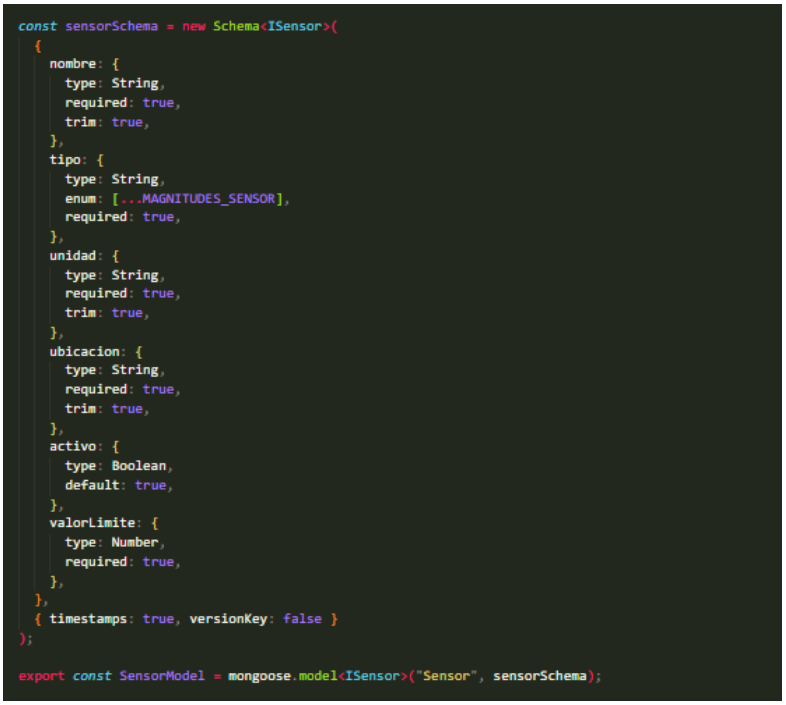
Campo	Tipo	Notas
email	String	Obligatorio, único, en minúsculas
password	String	Hash bcrypt; select: false, nunca se devuelve
fullName	String	Obligatorio
role	administrador | usuario	Por defecto usuario

```
12
13  const UserSchema = new Schema<IUser>(
14    {
15      email: {
16        type: String,
17        required: true,
18        unique: true,
19        lowercase: true,
20        trim: true,
21      },
22      password: {
23        type: String,
24        required: true,
25        select: false,
26      },
27      fullName: {
28        type: String,
29        required: true,
30        trim: true,
31      },
32      role: {
33        type: String,
34        enum: [...HUERTO_ROLES],
35        default: "usuario",
36      },
37    },
38    { timestamps: true, versionKey: false }
39  );
40
41  export const userModel = mongoose.model<IUser>("User", UserSchema);
```

(figura 4, modelo de users)

sensors

Campo	Tipo	Notas
nombre	String	Obligatorio
tipo	temperatura | humedad | luz	Cada sensor mide una única magnitud
unidad	String	Por ejemplo % o °C
ubicacion	String	Punto del huerto
activo	Boolean	Por defecto true
valorLimite	Number	Umbral a partir del cual una lectura es anómala



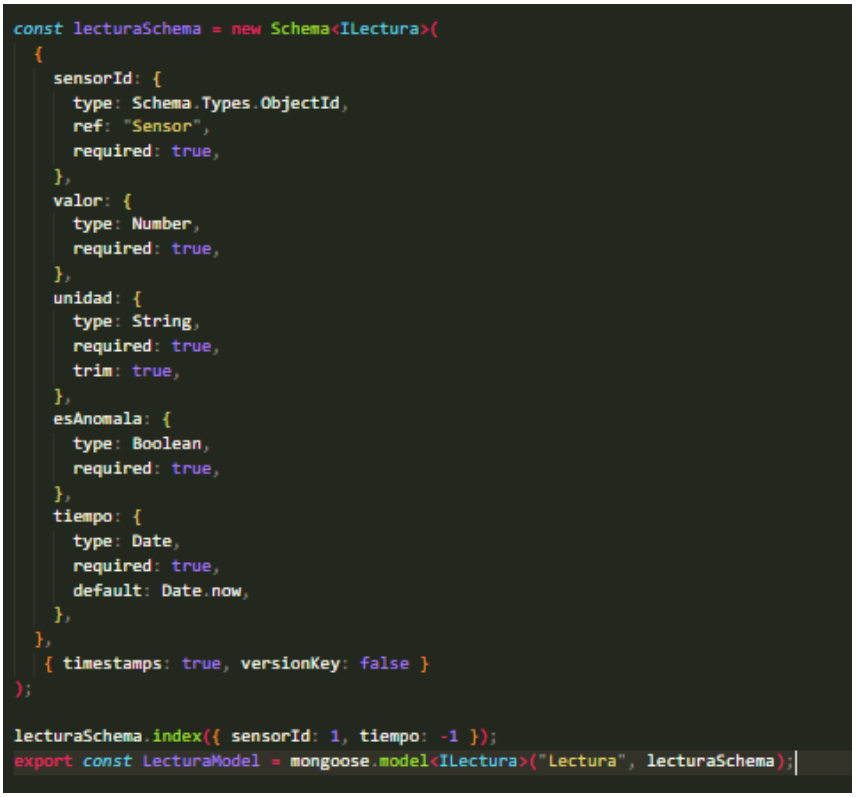
```
const sensorSchema = new Schema<ISensor>(
  {
    nombre: {
      type: String,
      required: true,
      trim: true,
    },
    tipo: {
      type: String,
      enum: [...MAGNITUDES_SENSOR],
      required: true,
    },
    unidad: {
      type: String,
      required: true,
      trim: true,
    },
    ubicacion: {
      type: String,
      required: true,
      trim: true,
    },
    activo: {
      type: Boolean,
      default: true,
    },
    valorLimite: {
      type: Number,
      required: true,
    },
  },
  { timestamps: true, versionKey: false }
);

export const SensorModel = mongoose.model<ISensor>("Sensor", sensorSchema);
```

(figura 5, modelo de sensores)

lecturas

Campo	Tipo	Notas
sensorId	ObjectId (ref Sensor)	Relación con el sensor
valor	Number	Medición
unidad	String	Unidad de la medición
esAnomala	Boolean	Calculado por el servidor al guardar
tiempo	Date	Instante de la medición (por defecto, el actual)



```
const lecturaSchema = new Schema<ILectura>(
  {
    sensorId: {
      type: Schema.Types.ObjectId,
      ref: "Sensor",
      required: true,
    },
    valor: {
      type: Number,
      required: true,
    },
    unidad: {
      type: String,
      required: true,
      trim: true,
    },
    esAnomala: {
      type: Boolean,
      required: true,
    },
    tiempo: {
      type: Date,
      required: true,
      default: Date.now,
    },
  },
  { timestamps: true, versionKey: false }
);

lecturaSchema.index({ sensorId: 1, tiempo: -1 });
export const LecturaModel = mongoose.model<ILectura>("Lectura", lecturaSchema);
```

(figura 6, modelo de lecturas)

Todas las colecciones llevan createdAt y updatedAt automáticos.

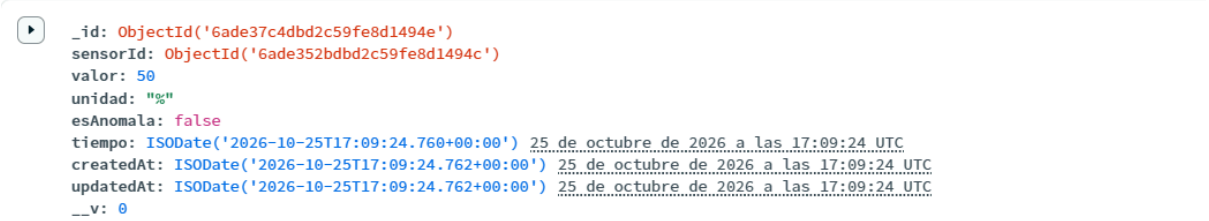
3.2 DECISIONES DE MODELADO

1. Referencia antes que duplicación:

La lectura guarda solo sensorId**.** El tipo de magnitud no se copia a la lectura, para filtrar por tipo se buscan los sensores de ese tipo y se filtra por sus identificadores. Así, si un sensor cambia no hay datos duplicados que se queden desactualizados.

```
_id: ObjectId('6ade352bdbd2c59fe8d1494c')
nombre: "Humedad invernadero"
tipo: "humedad"
unidad: "%"
ubicacion: "Invernadero"
activo: true
valorLimite: 80
createdAt: ISODate('2026-10-25T16:58:19.220+00:00') 25 de octubre de 2026 a las 16:58:19 UTC
updatedAt: ISODate('2026-10-25T16:58:19.220+00:00') 25 de octubre de 2026 a las 16:58:19 UTC
__v: 0
```

(figura 7, sensor guardado en MongoDB)



(figura 8, lectura guardada, su sensorId coincide con el id del sensor anterior)

2. esAnomala se guarda:

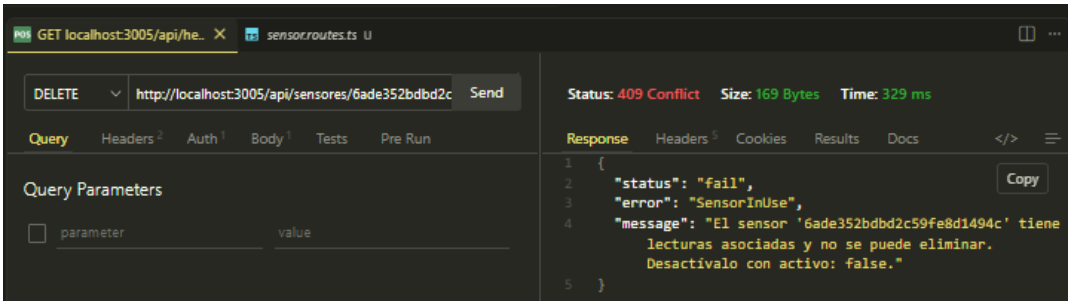
Es un dato derivado que permite filtrar y paginar directamente en la base de datos y hace que el histórico no cambie si después se modifica el límite del sensor (cada lectura conserva el veredicto que tuvo en su momento). Nunca lo envía al cliente, el esquema de entrada rechaza el campo.

3. Regla de anomalía:

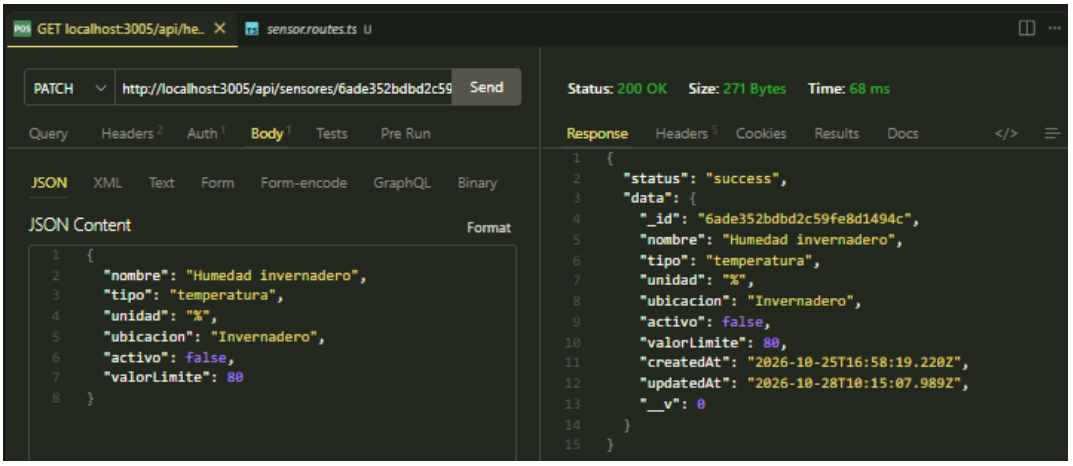
Una lectura es anómala si valor > valorLimite. Un valor exactamente igual al límite no lo es.

4. Integridad al borrar:

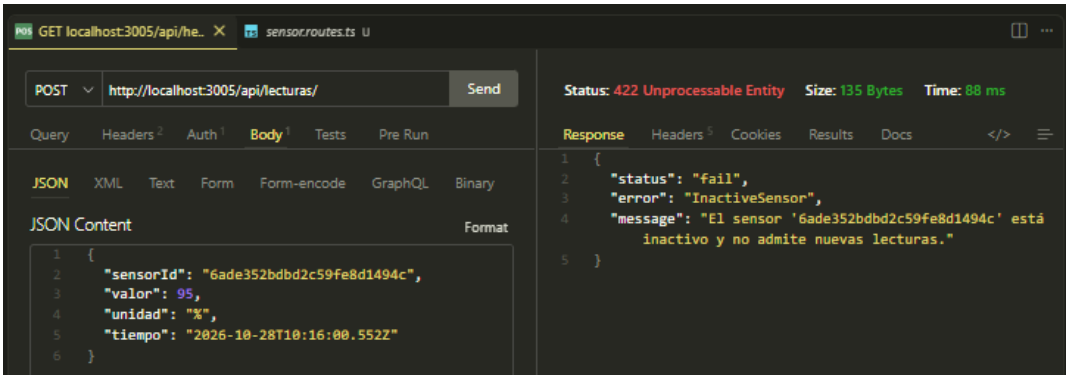
Un sensor con lecturas no se puede eliminar (409): evita lecturas huérfanas. Para dejar de usarlo se desactiva con activo: false, y un sensor inactivo no admite lecturas nuevas (422).



(figura 9, tratando de eliminar un sensor con lecturas)



(figura 10, cambiando el estado del sensor a falso para que no acepte nuevas lecturas)



(figura 11, intentando crear una nueva lectura en un sensor inactivo, resultado error 422)

4 DECISIONES DE DISEÑO REST

4.1 Recursos, rutas y códigos

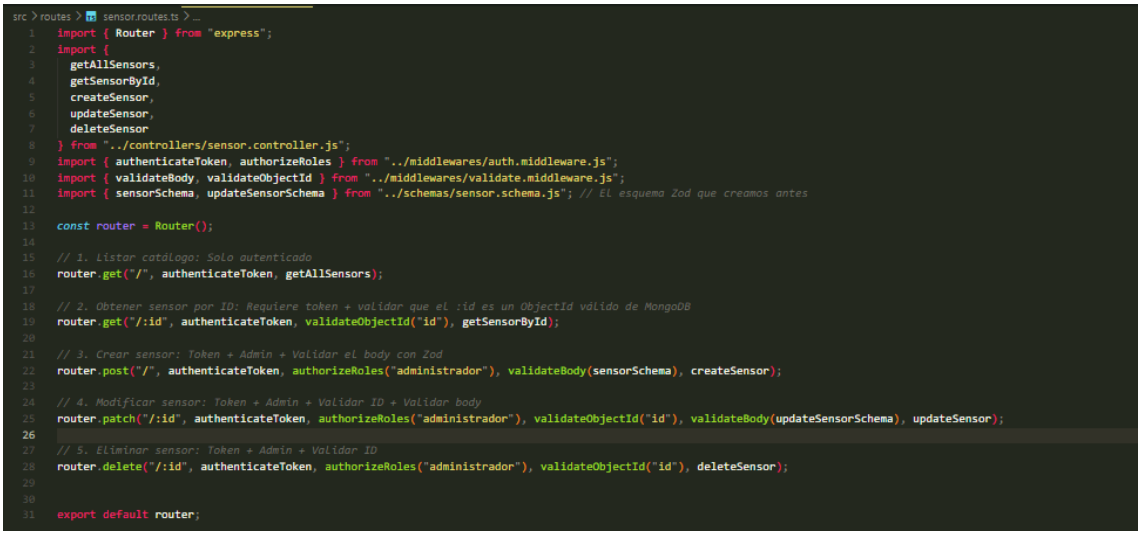
Método y ruta	Acceso	Éxito	Errores posibles
POST /api/auth/register	Público	201	400, 409
POST /api/auth/login	Público	200	400, 401

GET /api/sensores	Autenticado	200	401
GET /api/sensores/:id	Autenticado	200	400, 401, 404
POST /api/sensores	Administrador	201	400, 401, 403
PATCH /api/sensores/:id	Administrador	200	400, 401, 403, 404
DELETE /api/sensores/:id	Administrador	204	400, 401, 403, 404, 409
POST /api/lecturas	Autenticado	201	400, 401, 404, 422
GET /api/lecturas	Autenticado	200	400, 401

4.2 Justificaciones más importantes

Rutas, Los sustantivos en plural representan recursos (/sensores, /lecturas); la acción la expresa el verbo HTTP, no la URL. Los filtros, la paginación y la ordenación van en la query string.

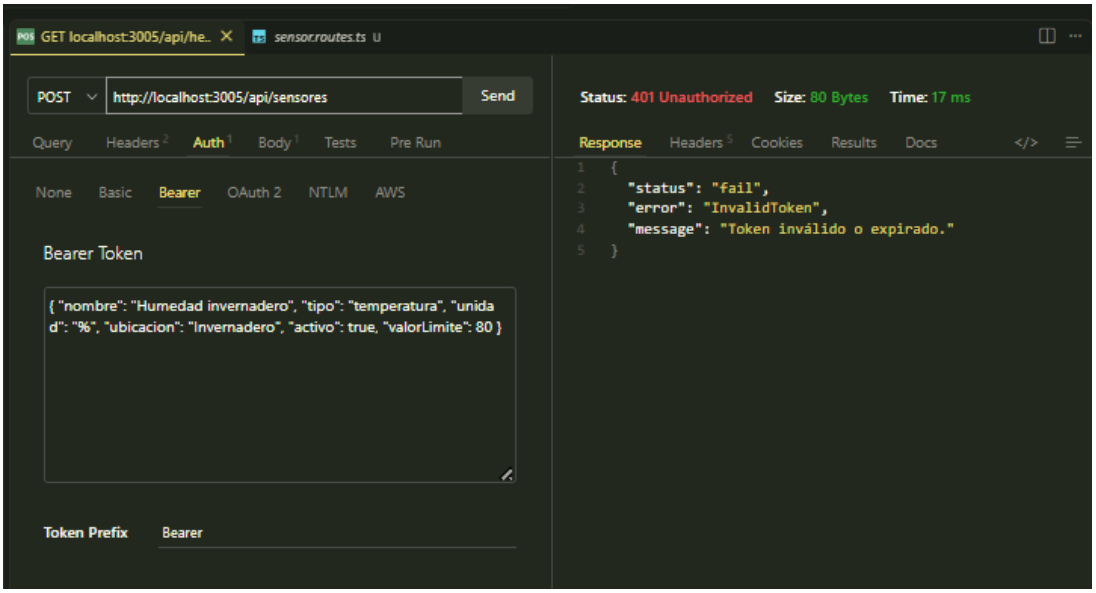
PATCH y no PUT, para modificar un sensor, se envían sólo los campos que cambian (al menos uno). PUT implicaría reemplazar el recurso completo.



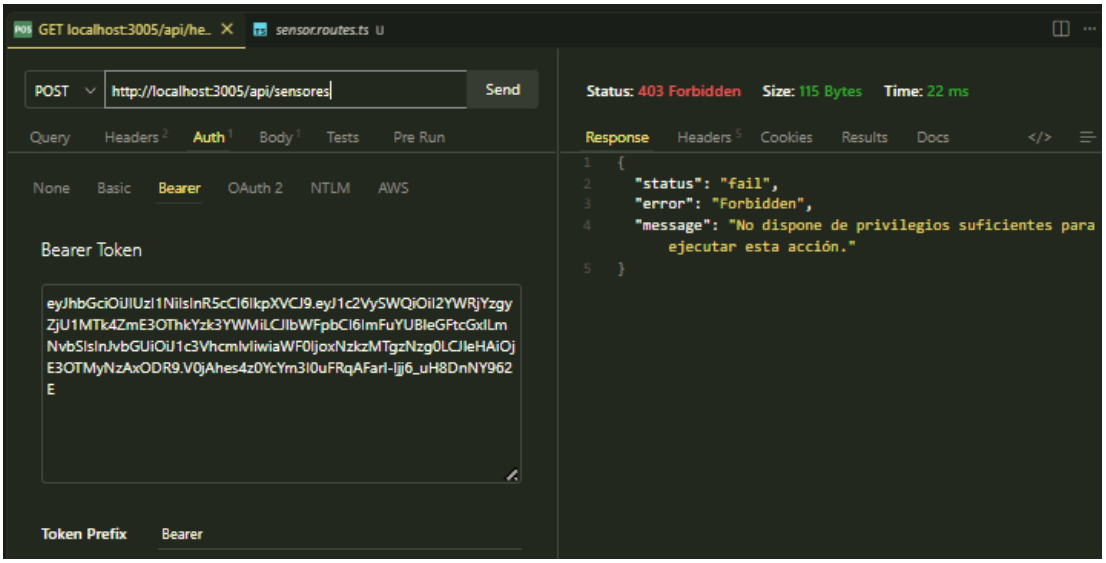
(figura 12, imagen de sensor.routes.ts)

201 al crear y 204 al borrar, 201 indica que se ha creado un recurso; 204 significa éxito sin contenido que devolver.

401 frente a 403, 401 cuando el sistema no sabe quién eres (sin token, token inválido o caducado); 403 cuando sabe quién eres pero tu rol no basta. Un usuario normal que intenta crear un sensor recibe 403, y la misma petición sin token recibe 401.



(figura 13 ,tratando de crear un sensor con un autentificador incorrecto, resultado: error 401)



(figura 14, tratando de crear un sensor con un autentificador correcto con permisos insuficientes, resultado: error 403)

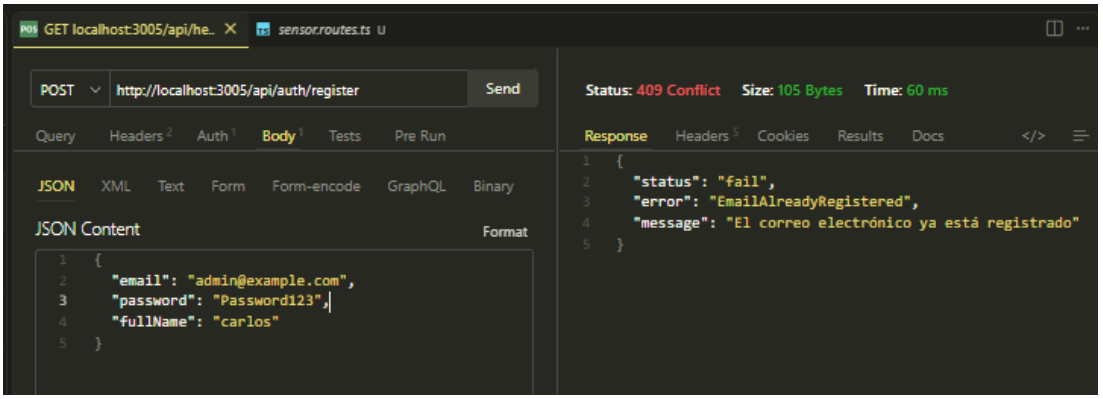
400 para entradas que no cumplen el formato (campos que faltan, tipos incorrectos, campos no permitidos, identificadores mal formados o JSON roto).

404 al registrar una lectura de un sensor inexistente, el sensor es una referencia dentro del cuerpo de la petición.



(figura 15, tratando de crear una lectura de un sensor inexistente)

409 para conflictos con el estado actual, un correo ya registrado o un sensor que no se puede eliminar porque tiene lecturas.



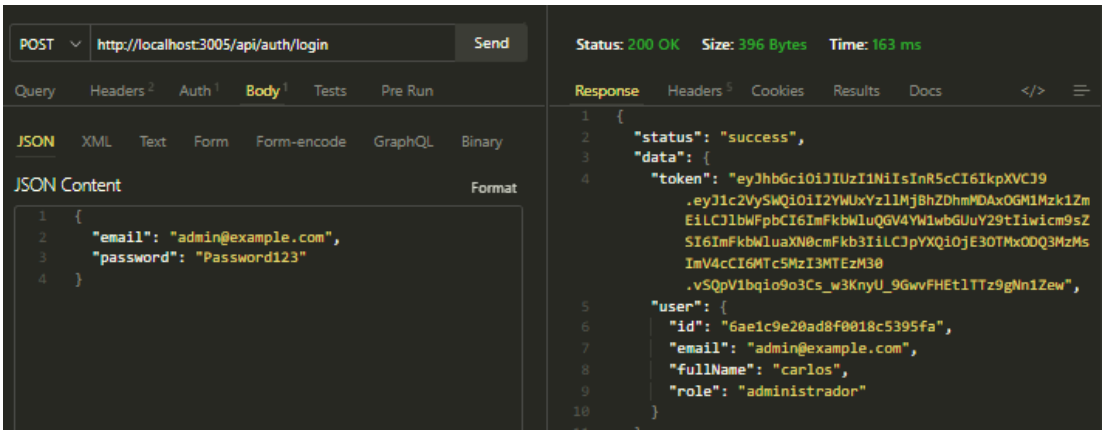
(figura 16, tratando de registrar un correo con un correo repetido)

5. AUTENTICACIÓN Y CONTROL DE ROLES

Alta y acceso, cualquier persona puede registrarse (usuario por defecto; el rol no se puede elegir porque el esquema de entrada rechaza campos desconocidos) y hacer login para obtener un token JWT.

Contraseñas: nunca en texto plano. Se guardan con bcryptjs y el campo está excluido de las consultas por defecto.

Login, si el correo no existe o la contraseña es incorrecta, la respuesta es idéntica (401 Invalid credentials), para no revelar qué correos están registrados.



(figura 17, login correcto, resultado 200 OK)

Token JWT firmado con HS256, con el identificador, el correo y el rol del usuario. El secreto y la caducidad se leen de variables de entorno. Al verificarlo se comprueba también la forma del contenido, no solo la firma.

```
let decoded: unknown;
try {
  decoded = jwt.verify(token, JWT_SECRET, { algorithms: ["HS256"] });
} catch {
  return next(new InvalidTokenError());
}
```

(figura 18 ,fragmento de código de auth.middleware.ts donde se decodifica algoritmo HS256)

Protección: Todas las rutas de sensores y lecturas exigen token. Las operaciones de administración añaden authorizeRoles("administrador"). El bloqueo se aplica en el servidor, nunca en el cliente.

```
import { Router } from "express";
import {
  getAllSensors,
  getSensorById,
  createSensor,
  updateSensor,
  deleteSensor
} from "../controllers/sensor.controller.js";
import { authenticateToken, authorizeRoles } from "../middlewares/auth.middleware.js";
import { validateBody, validateObjectId } from "../middlewares/validate.middleware.js";
import { sensorSchema, updateSensorSchema } from "../schemas/sensor.schema.js"; // El esquema Zod que creamos antes

const router = Router();

// 1. Listar catálogo: Solo autenticado
router.get("/", authenticateToken, getAllSensors);

// 2. Obtener sensor por ID: Requiere token + validar que el :id es un ObjectId válido de MongoDB
router.get("/:id", authenticateToken, validateObjectId("id"), getSensorById);

// 3. Crear sensor: Token + Admin + Validar el body con Zod
router.post("/", authenticateToken, authorizeRoles("administrador"), validateBody(sensorSchema), createSensor);

// 4. Modificar sensor: Token + Admin + Validar ID + Validar body
router.patch("/:id", authenticateToken, authorizeRoles("administrador"), validateObjectId("id"), validateBody(updateSensorSchema), updateSensor);

// 5. Eliminar sensor: Token + Admin + Validar ID
router.delete("/:id", authenticateToken, authorizeRoles("administrador"), validateObjectId("id"), deleteSensor);


export default router;
```

(figura 19, código de sensor.routes.ts, para hacer diferentes funciones como createSensor necesitamos autentificar nuestro rol)

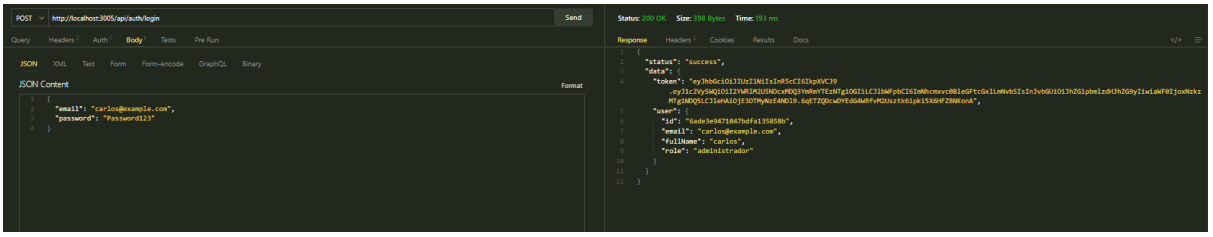
Primer administrador, no se pueden crear directamente usuarios con usuario administrador, para ello necesitaremos hacerlo directamente desde la base de datos en mongoDB

```
_id: ObjectId('6ade3e9471047bdfa135858b')
email: "carlos@example.com"
password: "$2b$10$xF.tP0L2IM5OjBOmAavLP.HWRy0G0tHXBNxS/gI5rFGh16oBY9XuO"
fullName: "carlos"
role: "usuario"
createdAt: ISODate('2026-10-25T17:38:28.982+00:00') 25 de octubre de 2026 a las 17:38:28 UTC
updatedAt: ISODate('2026-10-25T17:38:28.982+00:00') 25 de octubre de 2026 a las 17:38:28 UTC
__v: 0
```

(figura 20, estado en el que se encuentra Carlos en la base de datos con rol de usuario)



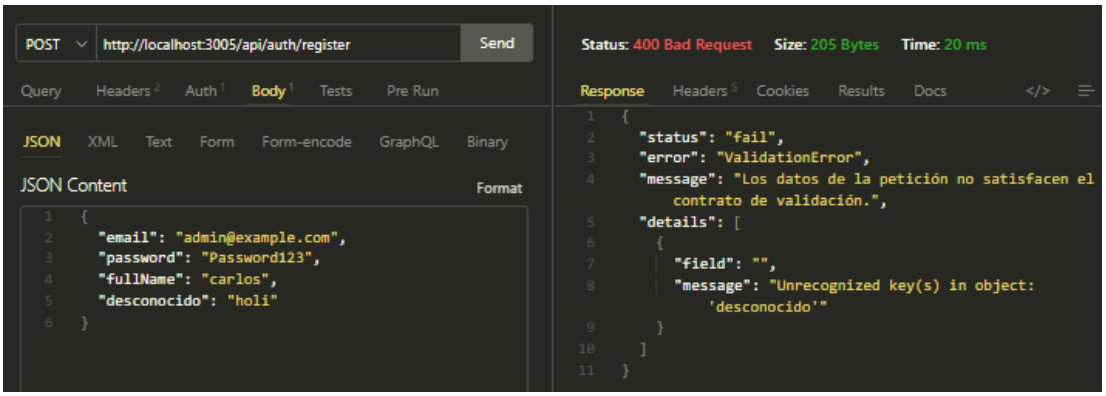
(figura 21, cambio de su nombre manualmente en la base de datos)



(figura 22, comprobación de carlos en la base de datos, ya como administrador)

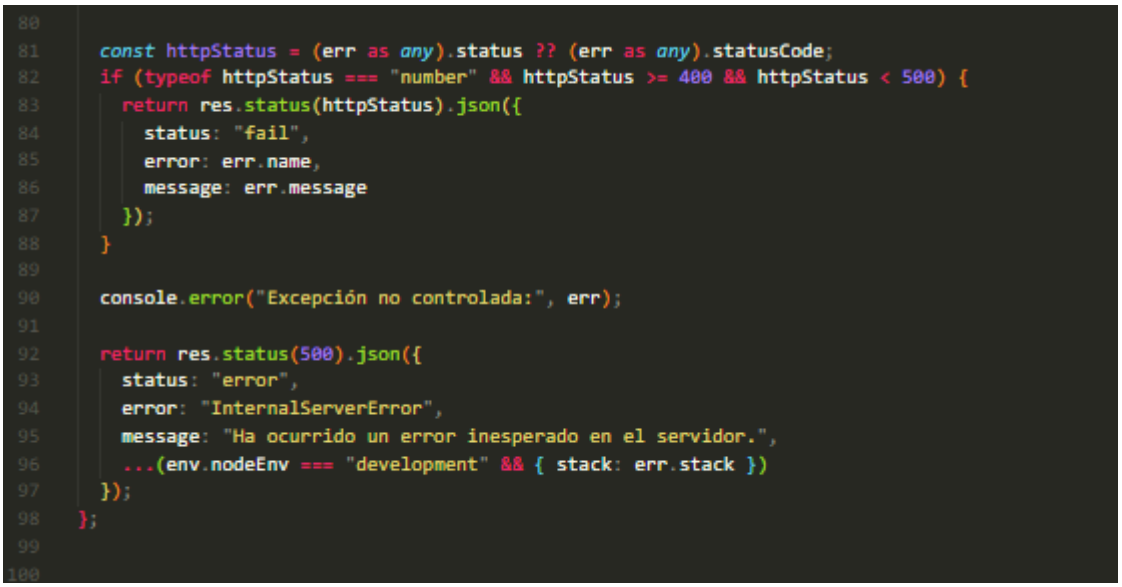
6. VALIDACIÓN Y MANEJO DE ERRORES

Validación con Zod en el borde de la aplicación: cuerpo (validateBody), consulta (validateQuery) e identificadores (validateObjectId). Los esquemas usan .strict() para rechazar campos desconocidos.



(figura 23, tratando de registrar un usuario con un nuevo campo desconocido)

Un único manejador de errores (globalErrorHandler) convierte cualquier fallo en una respuesta con el mismo formato. Traduce errores de Zod, JSON mal formado, errores propios (AppError y sus subclases), errores de Mongoose (validación, valores malformados, claves duplicadas) y errores del cliente de Express como un cuerpo demasiado grande (413). Lo inesperado devuelve un 500 genérico y se registra en el servidor sin exponer detalles.



```
80
81    const httpStatus = (err as any).status ?? (err as any).statusCode;
82    if (typeof httpStatus === "number" && httpStatus >= 400 && httpStatus < 500) {
83      return res.status(httpStatus).json({
84        status: "fail",
85        error: err.name,
86        message: err.message
87      });
88    }
89
90    console.error("Excepción no controlada:", err);
91
92    return res.status(500).json({
93      status: "error",
94      error: "InternalServerError",
95      message: "Ha ocurrido un error inesperado en el servidor.",
96      ...(env.nodeEnv === "development" && { stack: err.stack })
97    });
98  };
99
100
```

(figura 24, fragmento de código de error.middleware.ts, si después de verificar que no sea un error provocado por el cliente, como por ejemplo cuerpo demasiado grande, salta excepción no controlada 500)

7. PAGINACIÓN, FILTROS, ORDENACIÓN Y LÓGICA DE NEGOCIO

La API permite consultar las lecturas de los sensores de forma flexible mediante el endpoint GET /api/lecturas. Para facilitar la consulta de los datos, se han implementado diferentes parámetros que permiten controlar el número de resultados devueltos, filtrar las lecturas según determinados criterios y ordenar la información.

Los principales parámetros disponibles son:

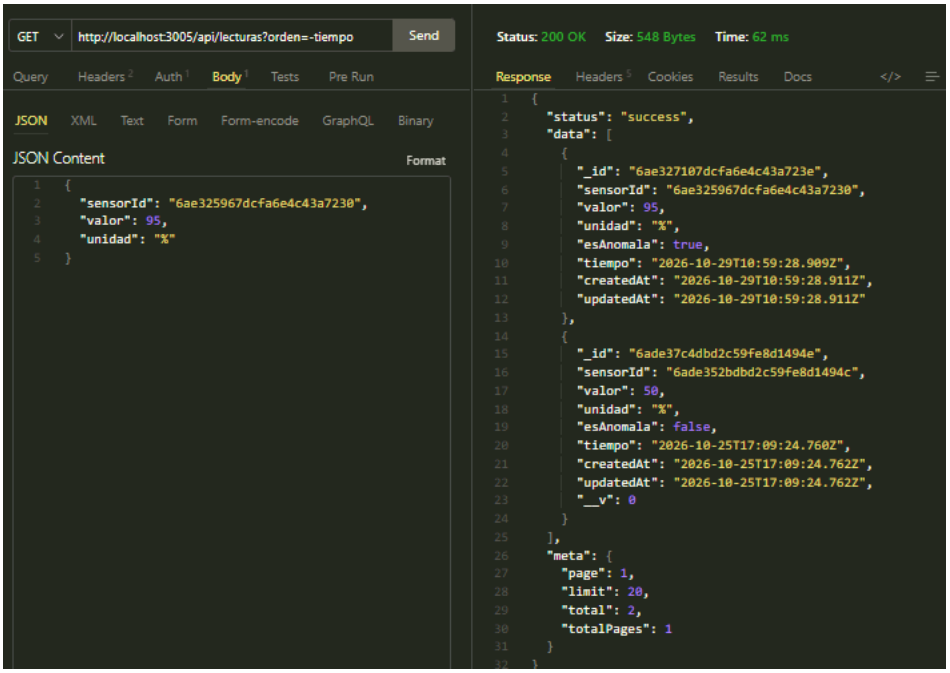
- **página**: indica la página de resultados que se quiere consultar (por defecto, 1
- **límite**: establece el número máximo de lecturas por página, con un máximo permitido de 100 (por defecto, 20).
- **sensorId**: permite obtener únicamente las lecturas asociadas a un sensor concreto.
- **tipo**: filtra por la magnitud que mide el sensor (temperatura, humedad o luz). Como la lectura no guarda el tipo, el servidor busca primero los sensores de ese tipo y filtra las lecturas por sus identificadores
- **anómala**: permite filtrar las lecturas según si se consideran anómalas (true) o normales (false).
- **orden**: permite ordenar los resultados por fecha (tiempo, -tiempo) o por valor (valor, -valor); el signo - indica orden descendente. Por defecto, -tiempo.

Estos parámetros se pueden combinar en una misma petición, lo que permite realizar consultas más específicas sin necesidad de crear diferentes endpoints para cada caso.

Por otra parte, la lógica de negocio relacionada con las anomalías se ejecuta en el servidor. Una lectura se considera anómala cuando su valor supera el límite establecido para el sensor al que pertenece (valor > valorLimite). Así, esta clasificación no depende del cliente y se mantiene un criterio uniforme en todas las consultas.

Por ejemplo usando el get: http://localhost:3005/api/lecturas?orden=-tiempo

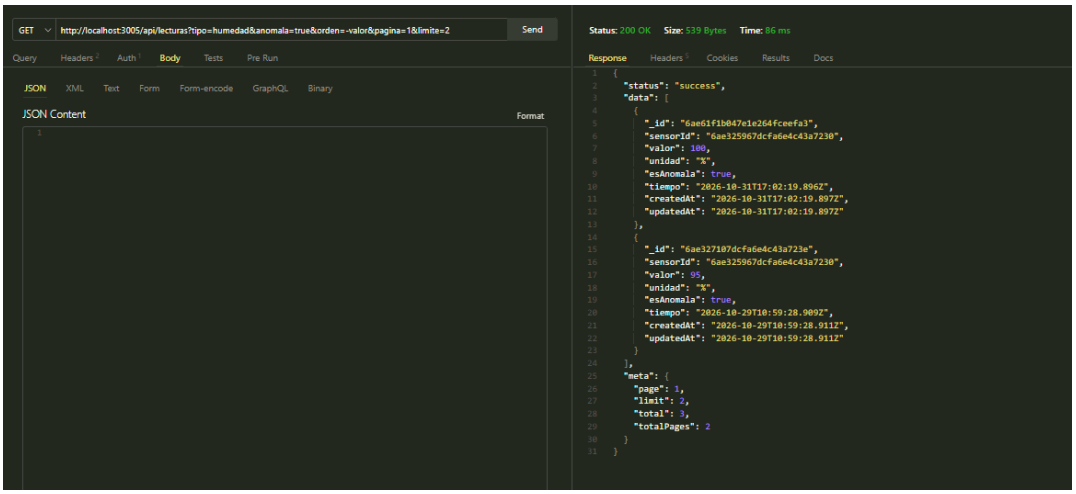
Se filtran las lecturas con fecha de orden descendente



(figura 25, representa lecturas en orden descendente)

Además un ejemplo de consulta que combina filtro, ordenación y paginación:

http://localhost:3005/api/lecturas?tipo=humedad&anomala=true&orden=-valor&pagina=1&limite=2

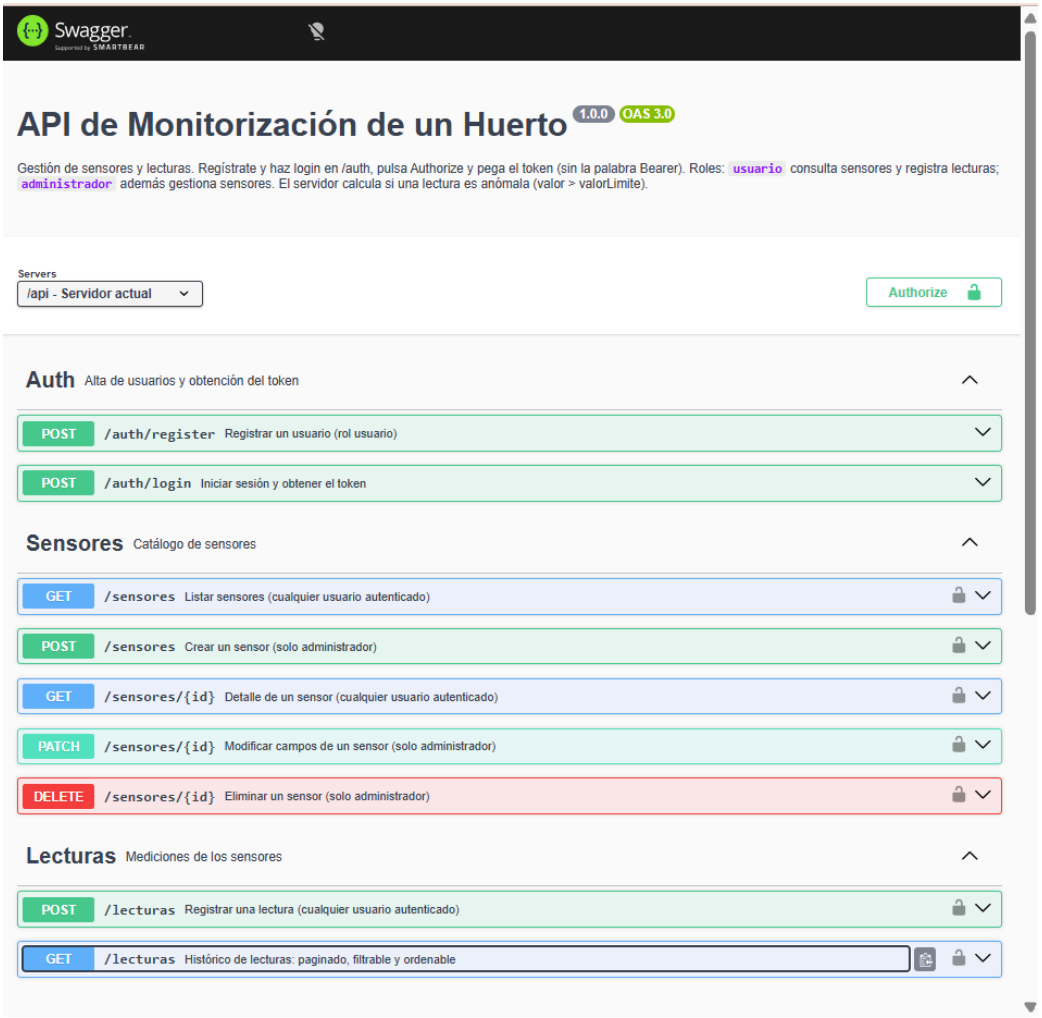


(figura 26,consulta combina filtros)

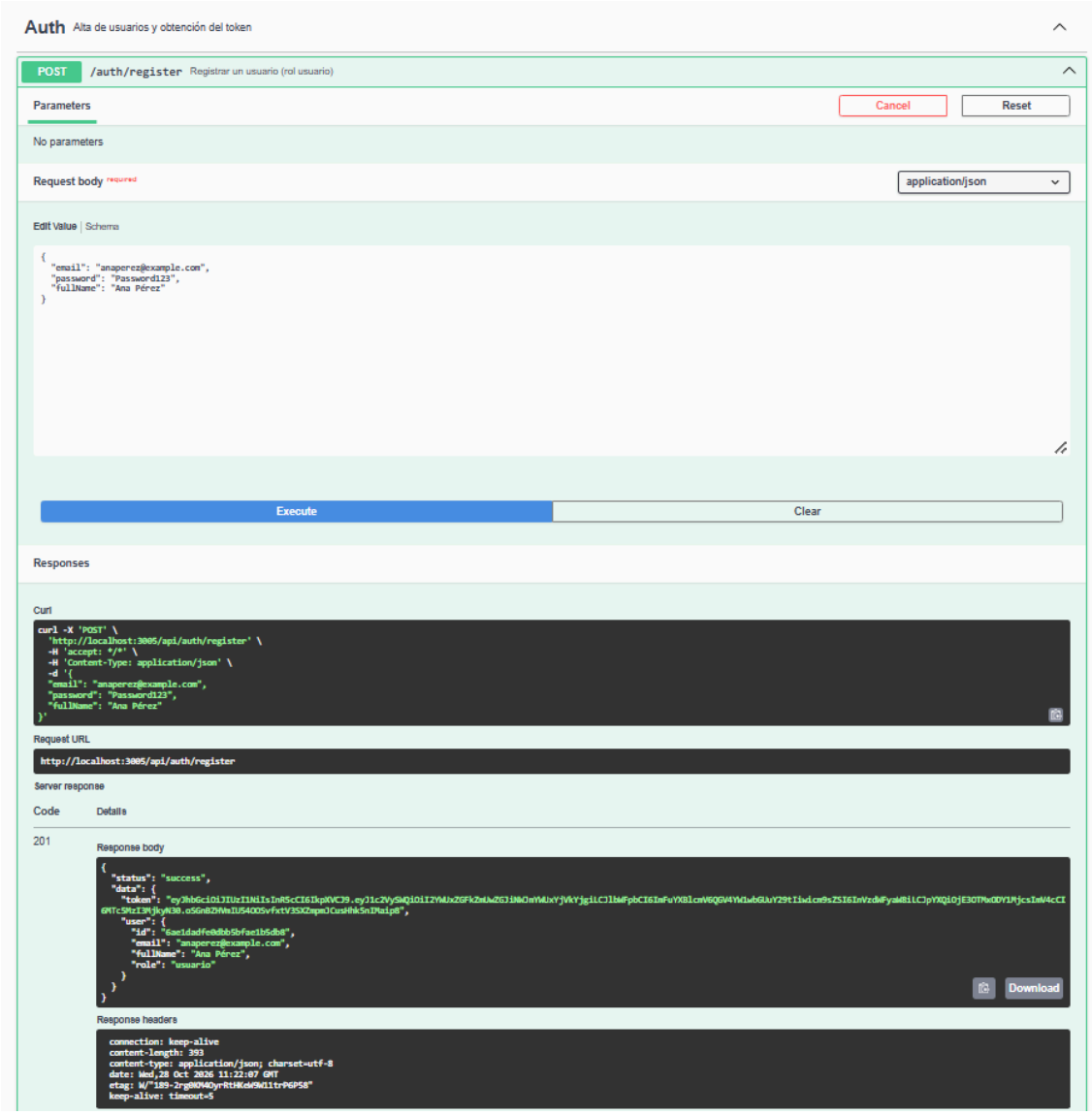
Devuelve únicamente las lecturas anómalas de sensores de humedad, ordenadas de mayor a menor valor, en páginas de 2 elementos. El objeto meta indica que se está en la página 1 de 2 y que hay 3 resultados en total, de modo que quien consume la API no tiene que adivinarlo.

8. DOCUMENTACIÓN OPENAPI

La especificación OpenAPI 3.0.3 está en src/docs/openapi.ts y se sirve con Swagger UI en /api/docs. Cubre autenticación, sensores y lecturas: qué recibe cada operación, qué puede devolver y en qué casos. Incluye el esquema de seguridad bearerAuth, de modo que el botón Authorize permite probar las rutas protegidas desde el navegador.



(figura 27, http://localhost:3005/api/docs/ en google)



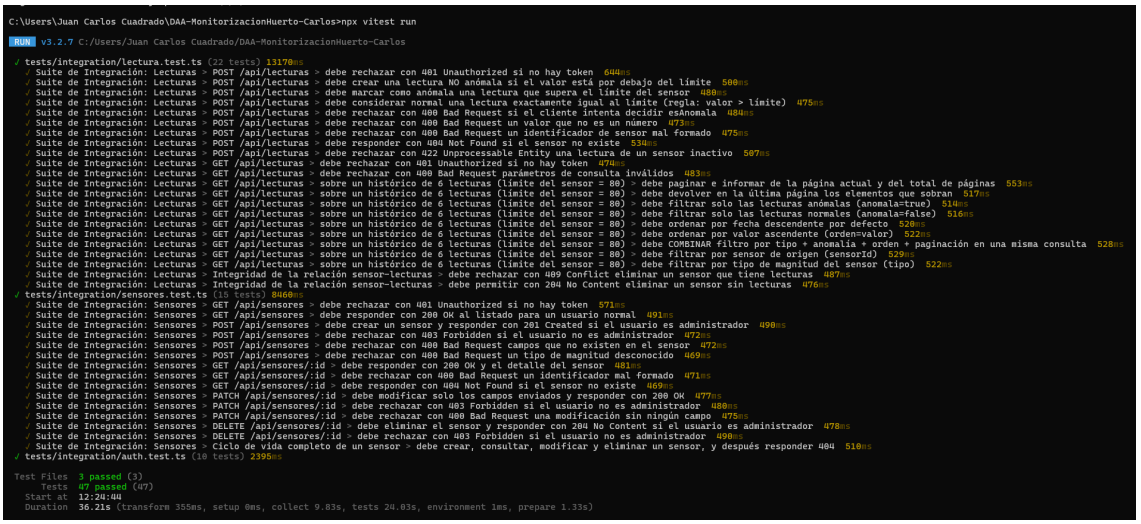
(figura 28, registrando un nuevo usuario en el sistema)

9. PRUEBAS AUTOMÁTICAS

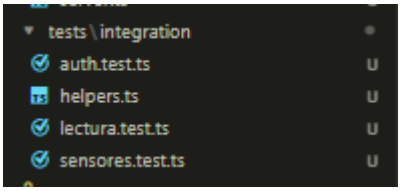
Test de integración con Vitest y Supertest sobre la aplicación completa (rutas, middlewares, controladores, servicios y bases de datos). Usan MongoDB en memoria (mongodb-memory-server), así no se tocan datos reales, y cada test parte de una base de datos vacía.

Fichero	Qué comprueba
auth.test.ts	Registro, contraseña cifrada, login correcto e incorrecto, mismo mensaje para correo y contraseña erróneos, rutas protegidas sin token y con token falso
sensores.test.ts	Bloqueo por rol (403) y sin token (401), validación, 404, y ciclo de vida completo: crear, consultar, modificar, eliminar y comprobar que ya no existe
lectura.test.ts	Cálculo de anomalía, sensor inexistente e inactivo, paginación, filtros y ordenación combinados, y integridad sensor-lecturas

Se ejecuta con npm test



(figura 29, pruebas de los test automáticos)



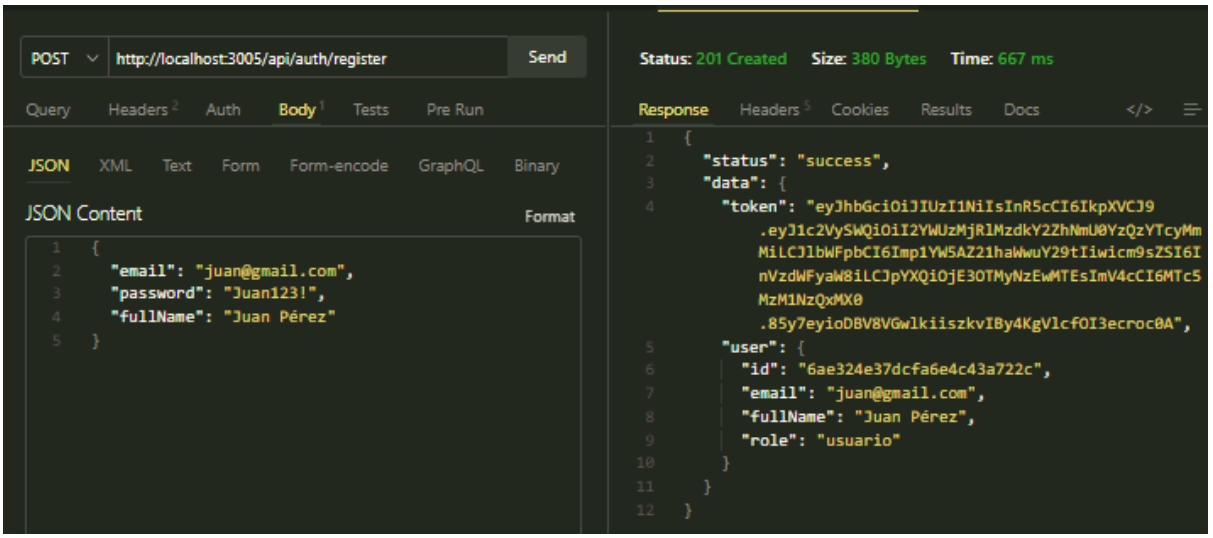
(figura 30, carpeta de test/integracion/)

helpers.ts no contiene tests, sino utilidades compartidas que evitan duplicar código en auth.test.ts, lectura.test.ts y sensores.test.ts. Sirve para gestionar una base de datos en memoria que se inicia, limpia y apaga en cada prueba para no tocar datos reales.

También facilita la autenticación registrando usuarios y devolviendo tokens con sus roles, y proporciona funciones para crear sensores de prueba rápida. Los archivos test.ts importan estas funciones para preparar el entorno y realizar peticiones con Supertest de forma limpia.

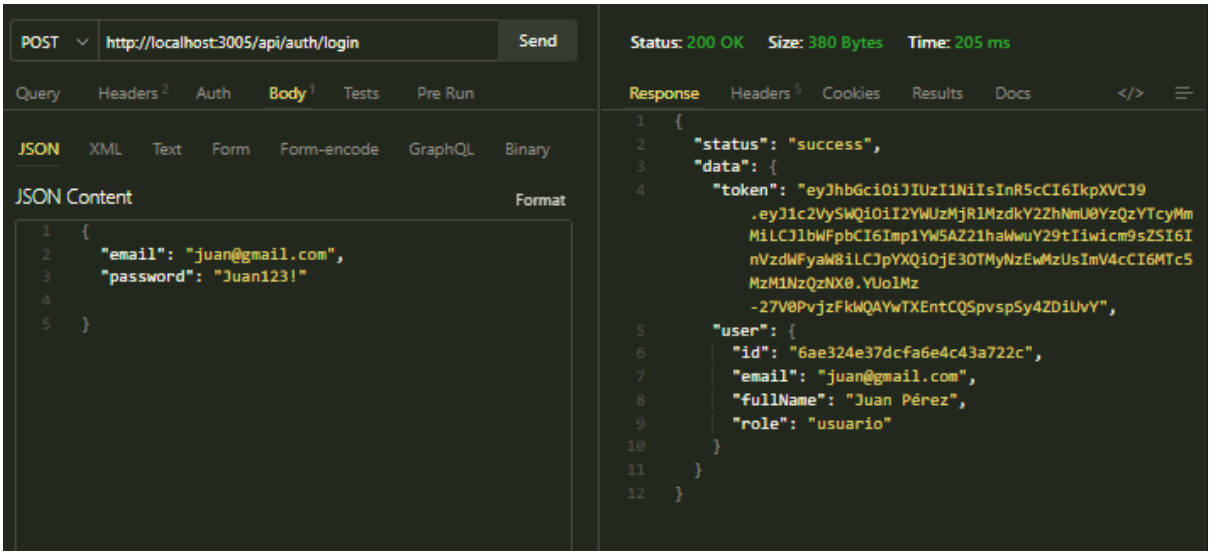
10. FLUJO COMPLETO CORRECTO

1. Registrar un usuario



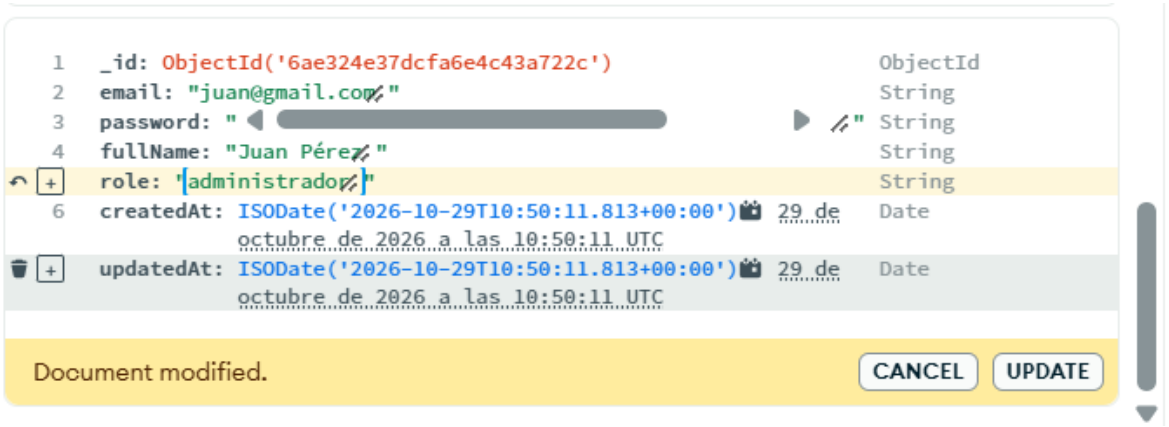
(figura 31, registrar usuario)

2. Login correcto

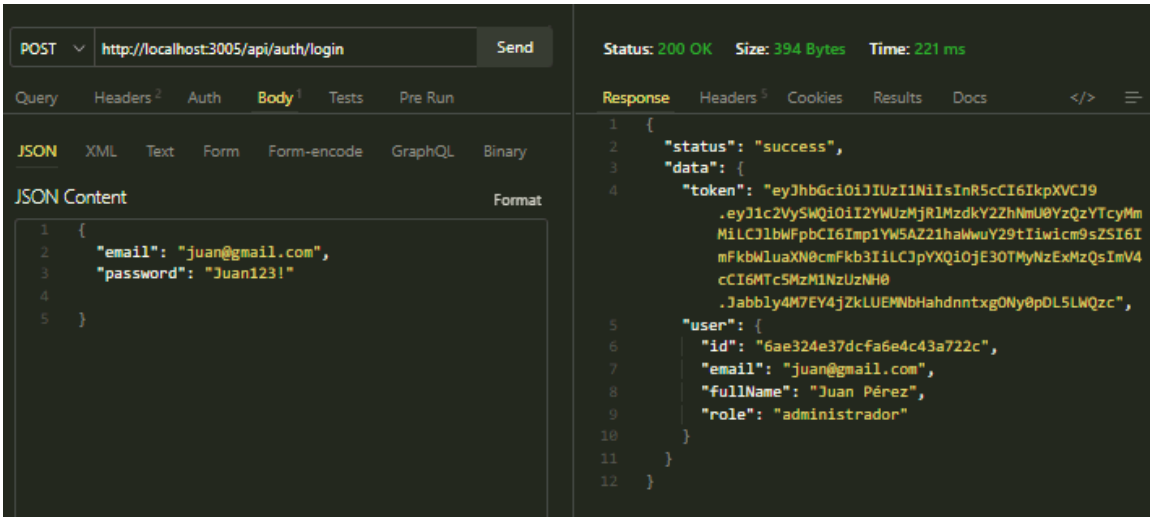


(figura 32, login usuario)

3. Hacer administrador

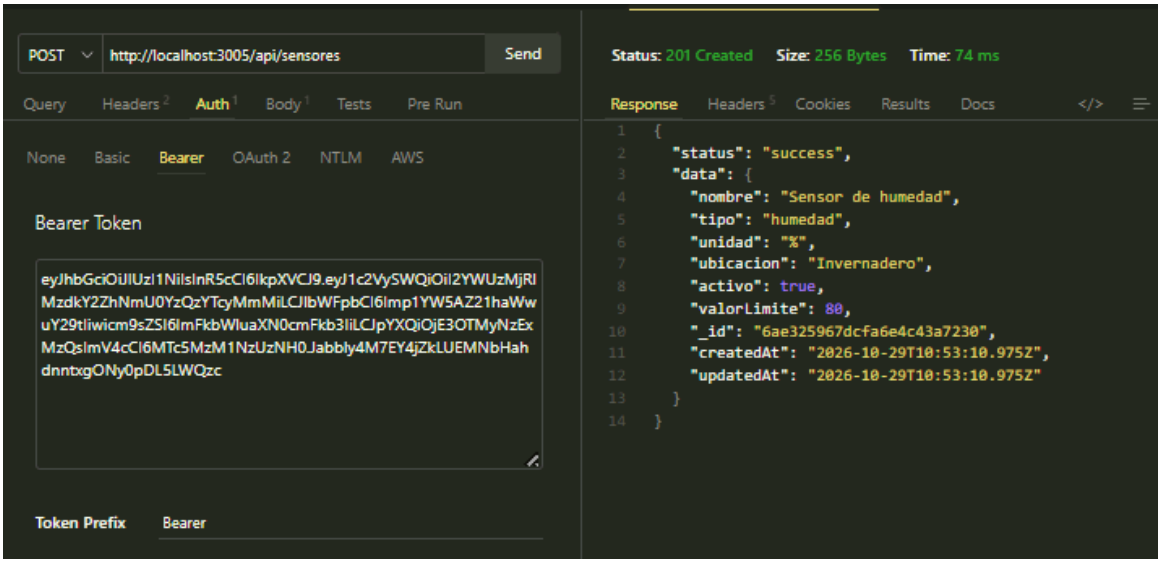


(figura 33, login creado en mongodb, ajustando su rol de forma manual)



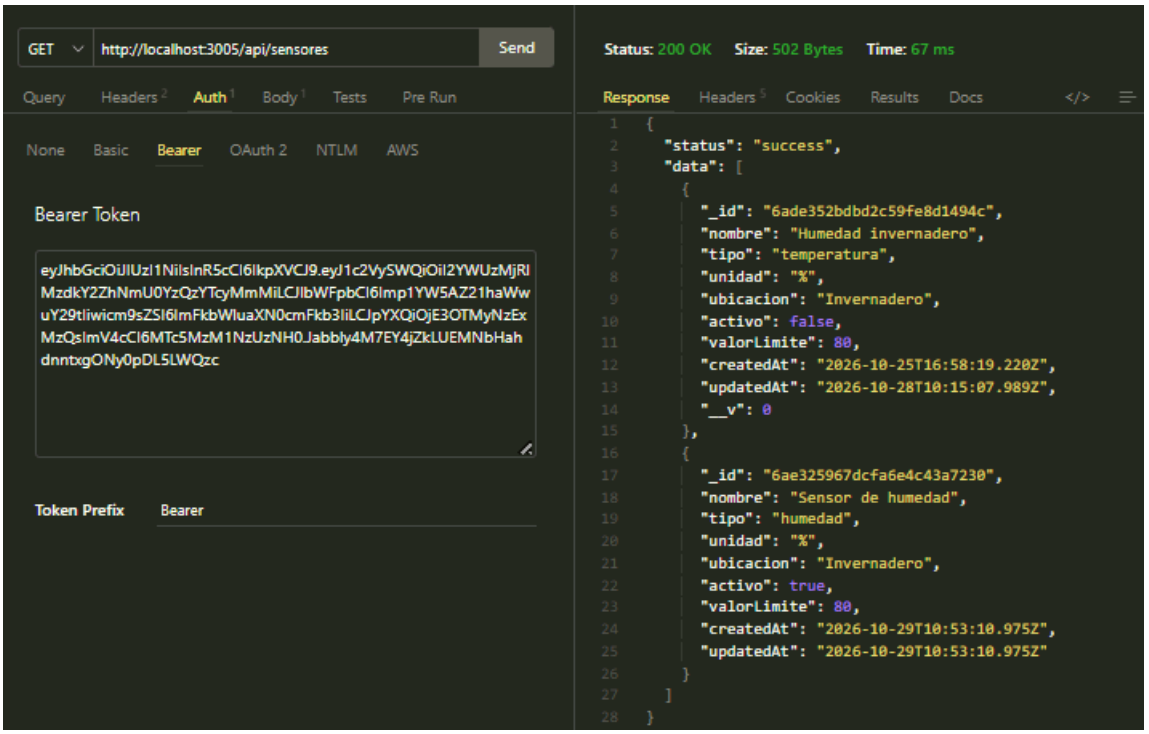
(figura 34, comprobar en thunder que al hacer login aparece el usuario con rol de administrador)

4. Crear sensor



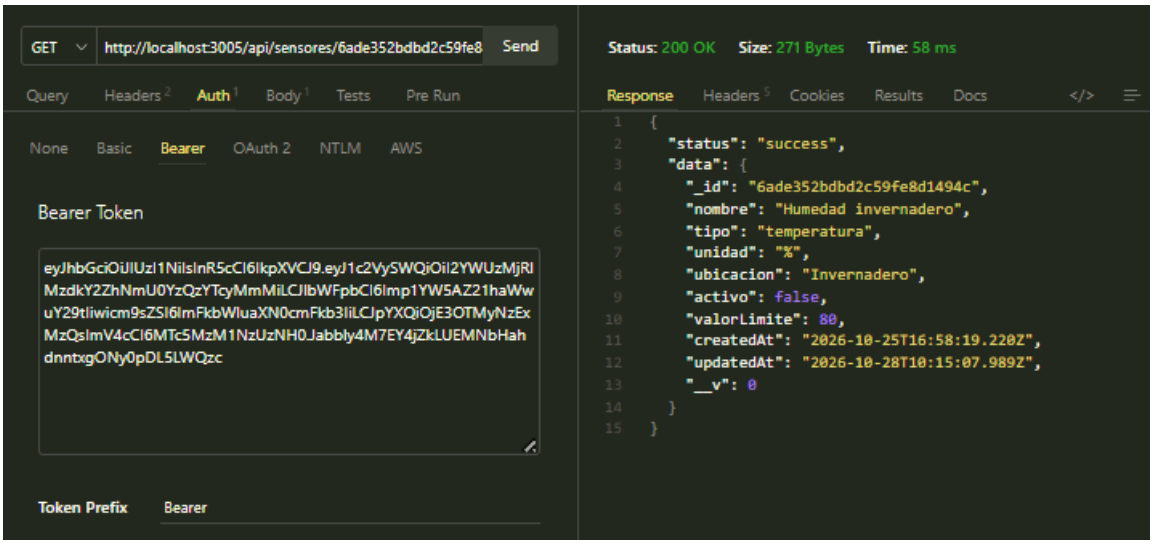
(figura 35, creando sensor)

5. Listar sensores



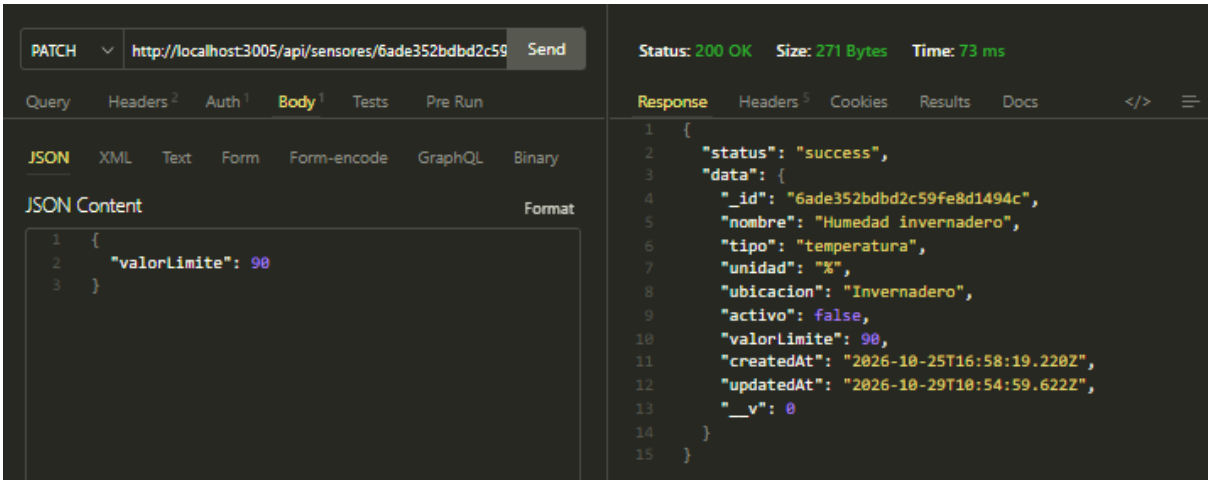
(figura 36, listar sensores)

6. Consultar un sensor



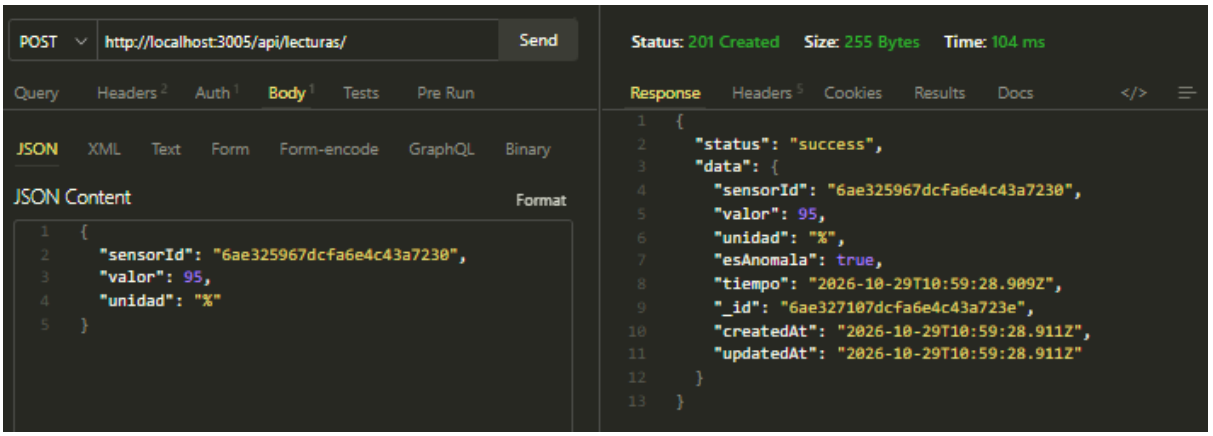
(figura 37, listar sensor concreto)

7. Modificar sensor



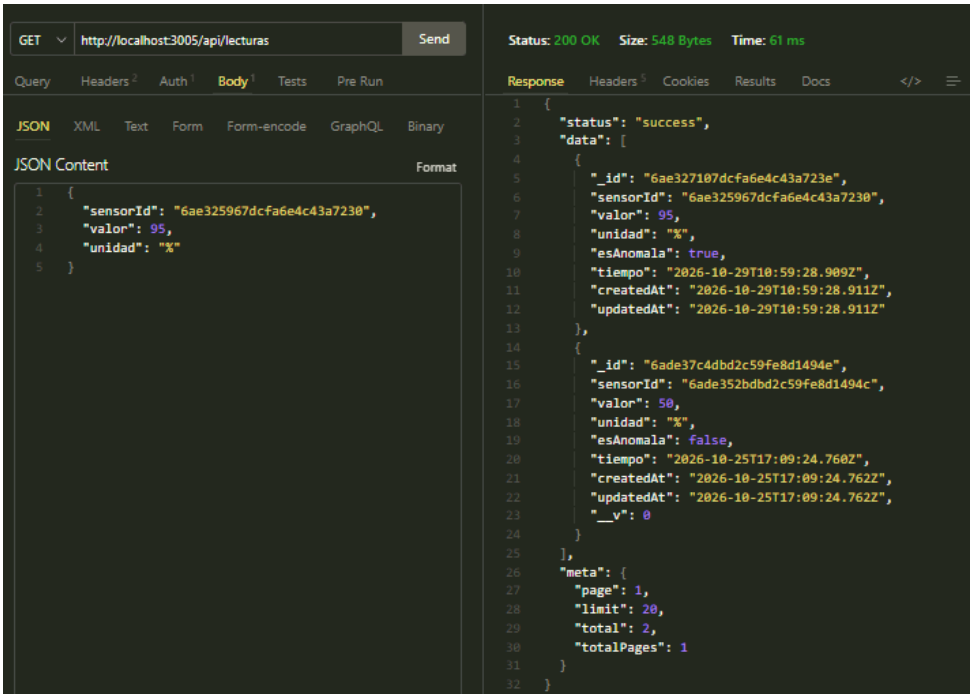
(figura 38, modificar sensor concreto)

8. Crear lectura



(figura 39, crear lectura concreta)

9. Listar lecturas



(figura 40, listar lecturas concreta)

11 DIFICULTAD TÉCNICA

1. Un documento de Mongoose no es un objeto plano.

Síntoma: TypeScript marcaba errores en auth.service.ts (user._id, buildAuthResult(user)), porque el tipo UserEntity (con id y passwordHash) no coincidía con lo que devuelven findOne() y create(): un documento de Mongoose con _id y password.

Causa: había dos representaciones del usuario, la pensada para objetos planos y el documento real de Mongoose. Además, quitar passwordHash con desestructuración sobre un documento no habría funcionado, con riesgo de devolver datos internos o el hash al cliente.

Solución: tipar con IUser (la interfaz del modelo) y construir la respuesta con una lista blanca explícita (toPublicUser: id, email, fullName, role).

2. Una regresión en la ordenación detectada por los tests.

Tras modificar la línea .sort(...) de lectura.service.ts, tres tests (orden por defecto, orden=valor y la consulta combinada) fallaron: las lecturas salían en el orden de inserción.

La solución fue volver a .sort(orden) y los 47 tests pasaron. Sin tests automáticos no lo habría visto: la API respondía 200 con datos plausibles.

12 VALORACIÓN PERSONAL

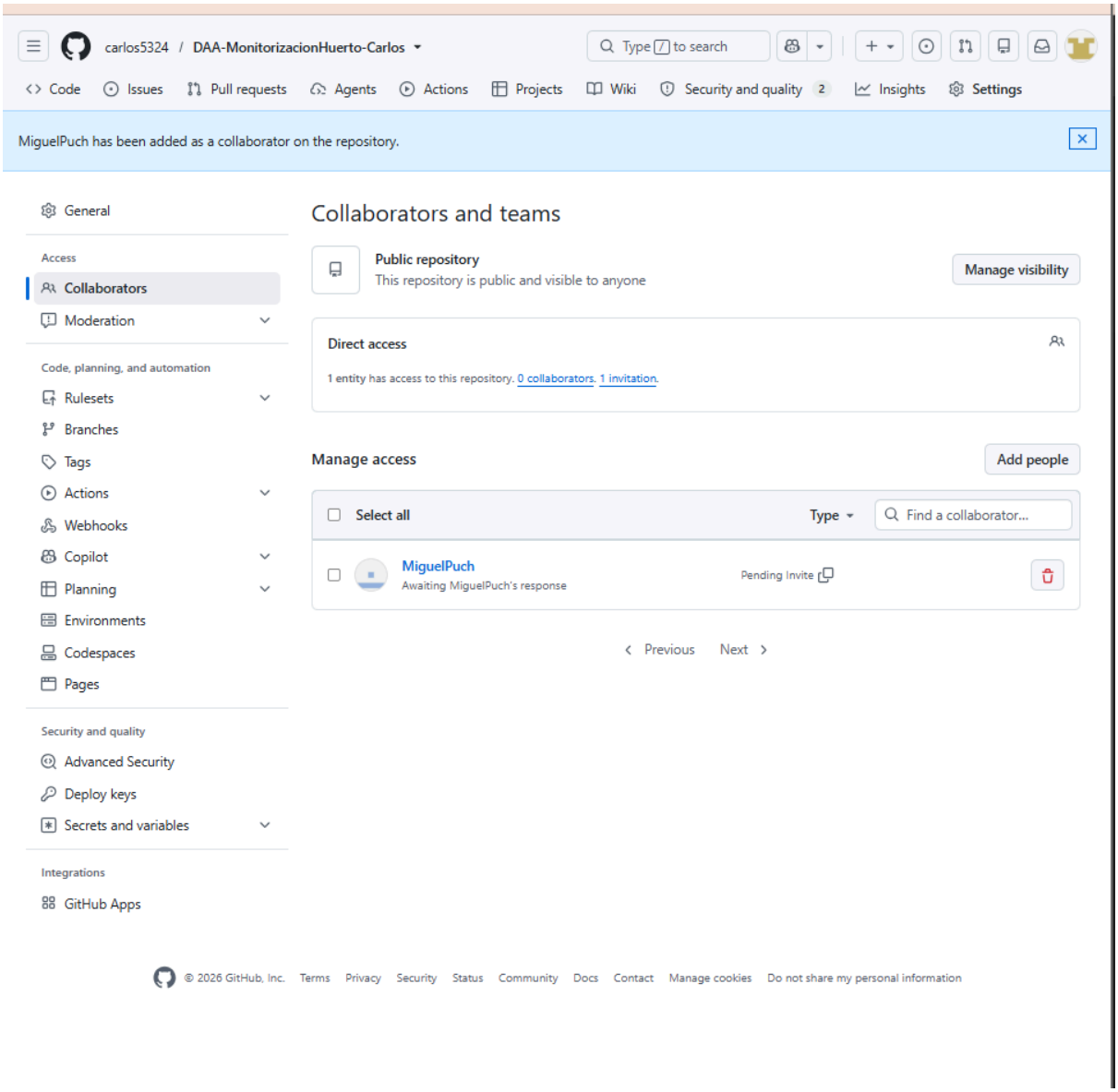
Personalmente, creo que este proyecto me ha servido bastante para aprender y afianzar conocimientos sobre el desarrollo de APIs REST. Al principio, algunas partes me parecían bastante complicadas, sobre todo entender cómo se relacionaban las rutas, los controladores, los servicios y los modelos, pero a medida que he ido avanzando he conseguido comprender mejor la estructura y el funcionamiento de la aplicación.

Una de las cosas que más valoro es haber podido trabajar con tecnologías como TypeScript, Express y MongoDB de una forma más práctica, aplicando conceptos como la autenticación, los permisos por roles, la validación de datos y las consultas con filtros y paginación. También he aprendido que probar una API no consiste únicamente en comprobar que una ruta responde, sino que hay que revisar los diferentes casos y los posibles errores.

Si tuviera que mejorar el proyecto, intentaría ampliar las funcionalidades y seguir mejorando las pruebas, además de revisar aspectos como la organización y el mantenimiento del código. Creo que siempre hay margen para hacer las cosas mejor.

13 COMPARTIR

Repositorio:https://github.com/carlos5324/DAA-MonitorizacionHuerto-Carlos
El profesor (MiguelPuch) ha sido invitado como colaborador del repositorio.


carlos5324 / DAA-MonitorizacionHuerto-Carlos
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality 2
Insights
Settings
MiguelPuch has been added as a collaborator on the repository.
General
Access
Collaborators
Moderation
Code, planning, and automation
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Collaborators and teams
Public repository
This repository is public and visible to anyone
Manage visibility
Direct access
1 entity has access to this repository. 0 collaborators. 1 invitation.
Manage access
Add people
Select all
Type
Find a collaborator...
MiguelPuch
Awaiting MiguelPuch's response
Pending Invite
Previous
Next
© 2026 GitHub, Inc.
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information